The Hong Kong Polytechnic University

Department of Computing

COMP4913 Capstone Project
Report (Final)

Financial Time-Series Prediction Using Advanced Neural Network Models

Student Name:	CHAN Cheung Hong
Student ID No.:	22081328D
Programme-Stream Code:	61435-FCS
Supervisor:	Dr Yujie Wu
Co-Examiner:	WANG XIANGMENG
2nd Assessor:	LIU Fiona
Submission Date:	10 April 2026

Abstract

This project benchmarks various sequence models for short-horizon forecasting along a unified reproducible pipeline which now includes support for multiple target definitions and both real (SPY) and synthetic (sine) data sources. The most recently archived final-report bundle (generated at 2026-03-31T20:31:21Z) assesses four tasks: sine_next_day, next_return, next_volatility, and next_mean_return. For each of these tasks, RNN, LSTM, GRU, Transformer and a flattened-sequence linear-regression baseline are compared on aligned splits using MSE/MAE and directional accuracy (DA).

The cross-task summary indicates that Baseline-LR outperforms all on sine_next_day and next_volatility, while LSTM and GRU are the best performers on next_return and next_mean_return, respectively. The best test MSE results were 4.8070e-08 (sine), 9.1979e-05 (next_return), 1.7733e-05 (next_volatility), and 1.5519e-05 (next_mean_return). These results support two main beliefs: (1) no single neural architecture is superior for all task definitions, and (2) good linear models are still an important component for financial forecasting benchmarks.

Abstract 2
List of Tables and Figures 6
Tables 6
Figures 6
Main body 6
Chapter 1. Introduction 6
1.1 Project background 6
1.2 Problem statement 6
1.3 Research aim 6
1.4 Objectives 6
1.5 Project Scope 7
1.6 Report Structure 7
Chapter 2. Background and Literature Context 7
2.1 Financial time-series forecasting 7
2.2. Traditional and machine-learning baselines 7
2.3. Recurrent neural networks 8
2.4 Improvement to the basic RNN 8
2.5 Application of Transformer networks to problems of sequencing 8
2.6 Where is the gap and where does my project fit 8
Chapter 3. Problem Statement and Objectives 8
3.1 Forecasting task definition 8
3.2 Research questions 9
3.3 Project objectives 9
3.4 Success criteria 9
Chapter 4. Data and Pre-processing 9
4.1 Data source and asset selection 9
4.2 Return definition 9
4.3 Sequence construction 10
4.4 Train/validation/test split 10
4.5 Normalisation and leakage prevention 10
Chapter 5. Methodology 10
5.1 Overall pipeline 10
5.2 Baseline model 11
5.3 RNN model 11
5.4 LSTM model 11
5.5 GRU model 11
5.6 Transformer model 11
5.7 Mathematical Formulation of Models 11
5.7.1 Baseline-LR (used in final benchmark) 12
5.7.2 Vanilla RNN (used in final benchmark) 12

5.7.3 LSTM (used in final benchmark)..... 12
5.7.4 GRU (used in final benchmark)..... 12
5.7.5 Transformer (used in final benchmark)..... 13
5.7.6 Classical models for context (not part of final benchmark)..... 13
5.8 Configuration source of truth..... 13
5.9 Training strategy..... 13
5.10 Evaluation metrics..... 14
Chapter 6. Experiment Design and Hyperparameter Adjustment..... 15
6.1 Why Hyperparameter Tuning is Necessary..... 15
6.2 Hyperparameter Tuning Policies..... 15
6.3 Scope of Models and Hyperparameters to be Tuned..... 15
6.4 Configurations that Yielded the Best Results..... 15
6.5 Validity Concerns About Hyperparameter Tuning..... 15
Chapter 7 - Results..... 16
7.1 Cross-task consolidated summary..... 16
7.2 Per-task tuned comparison highlights..... 16
7.3 Updated report artifact map..... 16
7.4 Prediction pattern visualization (updated figures)\..... 17
Task A — sine_next_day..... 17
Figure 4A-1 is included to illustrate optimization behavior and interpret model tuning quality across the simplest synthetic-style target..... 17
Figure 4A-2 is included to show which tuned model generalizes the best on the validation data for sine_next_day..... 18
Figure 4A-3 is included to assess the ranking of out-of-sample error for the held-out test split for Task A..... 18
Figure 4A-4 is included to determine whether stage-wise tuning had a material impact on the model performance in sine_next_day..... 19
Figure 4A-5 is included to analyze the best neural model's predictions to check for any calibration and bias..... 19
Figure 4A-6 is included to evaluate the quality of temporal fit in a localized region and investigate any lag/ overshoot that occurs..... 20
Task B — next_return..... 21
Figure 4B-2 is included to evaluate the generalization of validation before the final claims for Task B are tested..... 22
Figure 4B-3 is added to support the final model ranking for next_return on the held-out test data..... 23
Figure 4B-4 is included to analyze if the hyperparameter stages introduced meaningful improvements for each architecture..... 23
Figure 4B-5 is added to analyze the prediction calibration of the winning LSTM on the noisy signed returns..... 23
Figure 4B-6 is included to evaluate point prediction and sequence level fit for selected winner..... 24
Task C — next_volatility..... 25

Figure 4C-1 is included to assess the optimization behavior of the candidates for forecasting volatility..... 25
Figure 4C-2 is included to assess generalization quality for volatility predictions and to evaluate ranking validation.....26
Figure 4C-3 is for final held-out ranking submissions for next_volatility.....27
Figure 4C-4 is included to assess the impact of staged hyperparameter tuning on the outcomes of the volatility modeling.....27
Figure 4C-5 is included to assess the calibration of the best neural (GRU) predictions to the realised level of volatility for the time horizon..... 27
Figure 4C-6 is included to evaluate the performance of the best neural model in tracking volatility over time.....28
Task D — next_mean_return..... 29
Figure 4D-1 is included to compare training dynamics for moving-average return forecasting.....29
Figure 4D-2 is included to compare validation generalisation before reporting final test rankings..... 30
Figure 4D-3 is included to confirm held-out ranking for the smoothed return task..... 30
Figure 4D-4 is included to quantify hyperparameter-stage impact on final loss outcomes.....31
Figure 4D-5 is included to inspect calibration quality for the GRU winner on averaged signed returns.....31
Figure 4D-6 is included to check time-local alignment and lag behaviour for GRU predictions.....32
7.5 Summary of key findings..... 33
Chapter 8. Discussion.....33
Chapter 9. Limitations.....34
Chapter 10. Conclusion and Future Work.....34
Chapter 11. Project Contributions / What Has Been Achieved.....35
References..... 35
Appendices..... 36
Appendix A. Final tuned configurations (latest bundle highlights)..... 36

List of Tables and Figures

Tables

- Table 1. Cross-task consolidated summary.
- Table 2. Updated report artifact map.
- Table A-1. Final tuned configurations (latest bundle highlights).

Figures

- Figure 1. Training-loss comparison for the best tuned models and baseline.
- Figure 4A-1 to Figure 4D-6. Per-task training/validation/testing, hyperparameter impact, scatter, and prediction-slice visuals.

Main body

Chapter 1. Introduction

1.1 Project background

Predicting the correct answer is difficult because of unknown things in the market and because market data is often noisy and non-stationary. The predictions made about the market returns and return predictions have a high signal to noise ratio. This is even harder when trying to use machine learning and deep learning techniques. They have trying to use recurrent architectures and attention-based techniques in the deep learning space. [1], [5], [8]

This project is about a short-horizon forecasting system that is configurable and focuses on the target mode, the target horizon, and the target smoothing window. In the final archive, only one synthetic sanity task is present (sine_next_day) and three tasks for the SPY market (next_return, next_volatility, next_mean_return). The SPY is a good representation of the overall return and market trend for the U.S. equity markets, and the synthetic sine task in the task gives a check in a simpler regime to see that the pipeline is working properly. [9]

1.2 Problem statement

In this case, the problem isn't merely forecasting future events, but forecasting future events and determining if more complex neural sequence models give better results than a simpler baseline, at least for the case of using the same prepared dataset for training and testing. Financial forecasting is tougher than it seems. Models that look complex could actually be doing the same thing, which is capturing the underlying trend of the dataset and overfitting the noise. In that sense, a case of a fair shared-split comparison makes more sense in the academic world than showcasing a single model on its own. [1], [5]

1.3 Research aim

This project attempts to benchmark a few neural sequence architectures in the implemented task system, both in the synthetic sanity task and in the various definitions of the SPY target, to see if any of them outperform a collapsed sequence linear regression baseline in terms of prediction error and directional accuracy, given that they are using the same data splits and preprocessing.

1.4 Objectives

The specific objectives are as follows:

- To create a reproducible time series forecasting multi-task pipeline with real (SPY) and synthetic (sine) datasets.
- To assess RNN, LSTM, GRU, and Transformer models under the same experimental setup in different prediction tasks.
- To integrate a linear-regression baseline formed from the same lagged input features for relevant comparison.
- To implement structured hyperparameter tuning with validation-based criteria.
- To assess model performance based on error metrics and directional accuracy across tasks.

1.5 Project Scope

The most recent archive report bundle has been completed for four fully populated tasks: sine_next_day, next_return, next_volatility, and next_mean_return. The claims in this report are scoped to these definitions, and each task is tracked by explicit task_id, target_mode, horizon, and (if applicable) target_smooth_window.

1.6 Report Structure

The rest of this report is structured as follows: background literature, task definition, data preparation, methodology, tuning design, results, discussion, limitations, conclusion, and appendices

Chapter 2. Background and Literature Context

2.1 Financial time-series forecasting

Notable studies on financial forecasting focus on statistical and machine learning methods. A first useful distinction is between forecasting prices and forecasting returns. Returns tend to be more stationary and more appropriate for modeling short-term predictive relationships. Pricing is more difficult to project than return forecasting because predictive relationships are strong. Pricing structures are unpredictable and contain more noise than returns. Pricing structures tend offer stronger trends and contain scale effects. [5], [8]

Previous studies have demonstrated that deep learning models are capable of learning non-linear patterns for these types of problems, especially for financial time series forecasting. However, the relevance of utilizing deep learning models is highly dependent on the specific context (i.e. market, target variable and structure of the evaluation). For instance, Fischer and Krauss demonstrated that while LSTM (Long Short Term Memory) Networks) are capable of predicting outcomes in financial markets, the models tend to be fragile under realistic forecasting conditions. [1]

2.2. Traditional and machine-learning baselines

Linear models are considered a strong time-series forecasting baseline. Simplicity of the linear model also makes it useful for offering a baseline for more complex models. A more sophisticated model that does not outperform linear models is understood to not be forecasting well. Forecasting is understood to be close to that task's upper bound or that it was a weak forecasting signal if a neural network marginally outperforms linear regression. [5] [10]

2.3. Recurrent neural networks

A recurrent neural network (RNN) is a model that processes input sequentially while updating an internal hidden state.

The natural choice of architectural baseline of forecasting task is the simple vanilla RNNs. standard RNNs have the problem of the gradient problem running through the lengths of the sequential data and have problems of being able to remember things over the longer temporal windows

2.4 Improvement to the basic RNN

Vanilla RNNs do not have any inherent use of functional forms, or do not have, the ability to determine and process many temporal dependencies and have problems of be simplistic, or of using only simple, functional forms to structure the problem. LSTMs have use of memory cells and themselves allow the problems of temporal dependencies to be longer time horizons. [2] GRUs are are not at all dissimilar to LSTMs, however, GRUs have a parameterization which is simpler, GRUs do not have the computational complexity of LSTMs but still do have a computational complexity which is not at all trivial. In the forecasting of the financial markets, these more sophisticated forms of neural networks do have a necessary and advantageous, or a simplistic, RNN forms of neural networks do offer. A simpler and a more sophisticated, with the more sophisticated forms of neural networks, offer a more sufficient and a more appropriate. more simple and a more complex forms of neural networks.

2.5 Application of Transformer networks to problems of sequencing

transformers do not have the inherent use of functional forms, or do not have, the ability to determine and process many temporal dependencies and have problems of be simplistic, or of using only simple, functional forms to structure the problem.

2.6 Where is the gap and where does my project fit

In defining the boundaries of the trading systems, the project does not make the claim of trading systems superiority. This specific project, compared outcomes of the models-RNN, LSTM, GRU and transformers and the other randomized outcome which was a linear regression but all of the models were aligned and engineered as best as possible working integrated models with respect to the processing of the models and the evaluation of the models. Most of the effort here stems from the working and defined tasks which are the synthetic pure sanity check with sine_next_day which is stated among the market target the SPY next_return, next_volatility, next_mean_return. This specific target defined the bounds and the target smooth window and the target mode which is of any random return over shooting and overly generalizing the conclusions.

Chapter 3. Problem Statement and Objectives

3.1 Forecasting task definition

This forecasting system applies supervised learning on multifeature sequences. Let $f_t \in \mathbb{R}^8$ denote the engineered feature vector at day t. Each sample uses a rolling lookback window:

$X_t = [f_{t-L+1}, ..., f_t]$,

where L is the sequence length (seq_len, tuned per task/model). The repository accommodates multiple targets through the parameter target_mode:

- sine_next_day (synthetic sanity task),
- next_return (1-step return),
- next_volatility (forward rolling volatility),
- next_mean_return (forward rolling mean return),
- plus default/general modes like horizon_return.

The most recent final-report archive assesses the following four task IDs:

- sine_next_day (data_source=sine, horizon=1, target_smooth_window=1),
- next_return (data_source=spy, horizon=1, target_smooth_window=1),
- next_volatility (data_source=spy, horizon=1, target_smooth_window=5),
- next_mean_return (data_source=spy, horizon=1, target_smooth_window=5).

3.2 Research questions

The report attempts to answer these research questions:

- Which neural architecture, evaluated task-by-task (sine_next_day, next_return, next_volatility, next_mean_return), performs the best within the shared pipeline?
- How do model rankings vary between the synthetic sanity task (sine_next_day) and the SPY market tasks (next_return, next_volatility, next_mean_return)?
- Within each task definition, does a tuned neural model surpass the linear-regression baseline, and particularly, does this hold true for more than just a single return target?
- Is a lower MSE associated with improved directional accuracy across these task definitions?

3.3 Project objectives

The main technical aim is to create an apples-to-apples comparison by standardizing data preparation, split boundaries, evaluation metrics, and validation-focused model selection.

3.4 Success criteria

A model is deemed successful if it shows validation and test error is lower than all other alternatives in the same setup. Given that financial forecasting is often used for directional purposes, directional accuracy is reported as a metric in addition to MSE.

Chapter 4. Data and Pre-processing

4.1 Data source and asset selection

The most recent report bundle analyzes both synthetic and market tasks. For market tasks, data is downloaded from Yahoo Finance using the yfinance library with the instrument set to SPY and the default repository parameter START=2005-01-01. In parallel, the sine_next_day task utilizes the repository's generated sine-series feature frame as a controlled sanity benchmark. [9], [11]

4.2 Return definition

The raw downloaded close series are converted to daily log returns:

$$r_t = \log(P_t / P_{t-1})$$

Using returns rather than prices minimizes the impact of scale and makes the target more suitable for short-horizon statistical learning. [5]

4.3 Sequence construction

For neural models, the repository constructs three-dimensional input tensors of the shape (N, seq_len, 8), where each sample comprises seq_len consecutive days of 8 engineered features along with a scalar target. The optimized search space includes seq_len ∈ {20, 30, 60} in src/tuning/main.py.

The 8 engineered features produced by build_spy_feature_frame are:

- log_ret
- oc_ret
- hl_range
- vol_chg
- ma_5_gap
- ma_20_gap
- volatility_5
- volatility_20

Target construction is governed by target_mode, horizon, and target_smooth_window. In the most recent archived bundle, the tasks being evaluated are sine_next_day, next_return, next_volatility (target_smooth_window=5), and next_mean_return (target_smooth_window=5).

To be transparent, the archived best-tuned comparisons use the following winning seq_len values by task (best test-MSE run per task):

Task	Best test-MSE winner	Winning seq_len
sine_next_day	Baseline-LR	30
next_return	LSTM	30
next_volatility	Baseline-LR	20
next_mean_return	GRU	30

4.4 Train/validation/test split

All tasks use a chronological rather than random split to avoid future data leaking into training. The same train/validation/test method is used uniformly across all four archived tasks in reports/final_report_tasks/20260331T192814Z.

4.5 Normalisation and leakage prevention

Input sequences are standardised via the StandardScaler that is trained on the training inputs only (then applied to the validation and test sets). This prevents information leakage from the training transformation pipeline that could bias the evaluation. [5], [10]

Chapter 5. Methodology

5.1 Overall pipeline

The same high-level pipeline is followed by all models.

1. Download historical data for SPY.
2. Calculate log returns for each day.
3. Construct aligned input sequences consisting of 8 features per step along with a target configured for a specific task which can be sine_next_day, next_return, next_volatility, or next_mean_return.
4. Chronologically split the data into training, validation, and test sets.
5. Scale the training inputs and fit the scaler.
6. Use validation-based checkpoint selection to train each model.
7. The runs archived at the end are compared using MSE, MAE, and directional accuracy.

The pipeline describes a clear deep learning methodology involving data preprocessing, model training, selection by validation, and final evaluation. [7]

5.2 Baseline model

The initial model runs a flattened-sequence linear regression. Like the neural models, it employs the multifeature seq_len window, but it reshapes each (seq_len, 8) sequence into a tabular representation (seq_len × 8 features) so the target alignment stays the same. This baseline assesses the value of using a nonlinear sequence model against a simple linear model over the same information. [5], [10]

5.3 RNN model

The vanilla RNN model includes a recurrent layer over the seq_len input sequence and a linear output layer that maps the last hidden state to one forecast value. This model is one of the simplest neural sequence models. [6]

5.4 LSTM model

The LSTM model uses gated memory units instead of the simple recurrent cell, and again uses the last sequence output to make one scalar prediction for one time step. Based on the literature, LSTM should be more effective at maintaining useful information across the lookback window. [1], [2]

5.5 GRU model

The GRU model is very similar to the LSTM model and uses update and reset gates, which result in fewer parameters. It is included in the study to determine whether a lighter gated recurrent architecture can be comparable to LSTM for this problem. [3]

5.6 Transformer model

The Transformer model first projects each timestep 8-dimensional feature vector into a learned embedding, adds positional encodings, and then applies several stacked Transformer encoder layers, using the last timestep's embedding for a scalar prediction. This model architecture is designed to verify whether self-attention can be more effective than recurrence in this multifeature financial sequence problem. [4]

5.7 Mathematical Formulation of Models

This section outlines the key equations related to the models discussed in this document and includes a brief symbol explanation to aid the reader.

The input for each instance consists of a rolling window of L days (where L = seq_len) and F = 8 features per day.

$X_t = [x_{t-L+1}, ..., x_t], \ x_{tau} \text{ in } R^F$

All models give a single scalar prediction as output, which is denoted as y_{hat_t}. The model is trained using the mean square error (MSE), defined as:

$MSE = (1/N) * sum_{t=1}^{N} (y_t - y_{hat_t})^2$

where N is the number of training samples, y_t is the true value that is targeted, and y_{hat_t} is the prediction from the model.

5.7.1 Baseline-LR (used in final benchmark)

The baseline model flattens the entire sequence into a single vector and performs a linear mapping:

$y_{hat_t} = w^T vec(X_t) + b$

Here, $vec(X_t)$ refers to the vectorization of all lagged features into a vector of length L*F. This results in a linear regression model that uses the same data as the neural models.

5.7.2 Vanilla RNN (used in final benchmark)

An RNN updates a hidden state in a sequential manner as follows:

$h_{tau} = tanh(W_x x_{tau} + W_h h_{tau-1} + b_h)$

$y_{hat_t} = W_o h_t + b_o$

The final hidden state captures the information for the entire lookback window and is then projected to the output.

5.7.3 LSTM (used in final benchmark)

Adding gated memory, LSTM has

$i_{tau} = sigmoid(W_i[x_{tau}; h_{tau-1}] + b_i)$
$f_{tau} = sigmoid(W_f[x_{tau}; h_{tau-1}] + b_f)$
$o_{tau} = sigmoid(W_o[x_{tau}; h_{tau-1}] + b_o)$
$g_{tau} = tanh(W_g[x_{tau}; h_{tau-1}] + b_g)$
$c_{tau} = f_{tau} * c_{tau-1} + i_{tau} * g_{tau}$
$h_{tau} = o_{tau} * tanh(c_{tau})$
$y_{hat_t} = W_y h_t + b_y$

i_{tau}, f_{tau}, and o_{tau} are input, forget, and output gates; c_{tau} is the memory cell.

5.7.4 GRU (used in final benchmark)

A lighter gated architecture is employed by GRU:

$$z_{\tau} = \text{sigmoid}(W_z[x_{\tau}; h_{\tau-1}] + b_z)$$
$$r_{\tau} = \text{sigmoid}(W_r[x_{\tau}; h_{\tau-1}] + b_r)$$
$$\tilde{h}_{\tau} = \tanh(W_h[x_{\tau}; r_{\tau} * h_{\tau-1}] + b_h)$$
$$h_{\tau} = (1 - z_{\tau}) * h_{\tau-1} + z_{\tau} * \tilde{h}_{\tau}$$
$$\hat{y}_t = W_y h_t + b_y$$

In this case, z_{τ} regulates the update strength and r_{τ} determines how much of the previous state should be retained.

5.7.5 Transformer (used in final benchmark)

Each encoder block utilizes scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

where $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. The output for the final sequence representation z_t is:

$$\hat{y}_t = W_h z_t + b_h$$

5.7.6 Classical models for context (not part of final benchmark)

In previous iterations, ARIMA/SARIMA and exponential smoothing were discussed. In the context of this project, the preferred empirical comparison is contained to the Baseline-LR, RNN, LSTM, GRU, and Transformer models, all of which are implemented within a common framework.

5.8 Configuration source of truth

The repository default (for example, HORIZON=1, TARGET_MODE=horizon_return) is set in src/common/config.py. Run-specific (including CLI) overrides such as --horizon and --target-mode are handled in src/common/runtime_config.py and, therefore, are passed on during experiment preparation. This way, archived experiments may be able to differ intentionally from default values.

In this report, archived final metrics are derived from runtime-resolved experiment configurations (effective CLI + staged winner selections). These metrics are not derived from the static default values set in src/common/config.py. Specifically, the run truth is derived from reports/final_report_tasks/20260331T192814Z/tuning_winners.csv, per-task reports/final_report_tasks/20260331T192814Z/*/best_tuned_comparison_*, and the bundle-level synthesis report final_report_tasks/20260331T192814Z/overall_task_summary.md.

5.9 Training strategy

The pipeline employs the Adam optimizer, mean squared error loss, early stopping, the best validation state is checkpointed, simple/smooth validation loss-based learning rate reduction, and a validation-driven approach to hyperparameter tuning. The final archived best-tuned comparison is from the frozen winners stored in tuning_winners.csv. [12]

Figure 1 is meant to show that the tuned pipeline provides stable convergence behavior before the report processes task-specific winners.

Figure 1. Training-loss comparison for the best tuned models and baseline.

- Tests whether model optimisation remains stable and comparable across architectures.
- All tuned models converge, but with distinct loss levels across families.
- Supports the report's model comparison framework and motivates more detailed analysis of task-level results.

5.10 Evaluation metrics

Three metrics reported:

- MSE: is the main optimisation/evaluation criterion. The lower the error of predictions, the better the score (penalises large forecast errors).
- MAE: is the average absolute error (provides a scale-consistent average).
- Directional Accuracy (DA) indicates the rate of correct sign predictions against a given return.
-

DA's significance lies in the financial realm as models can be off in the magnitude of returns yet still classify the move in the correct direction.

DA requires a specific target_mode for interpretation compared to an arbitrary cross-task assessment. For signed return targets (as an example next_return and next_mean_return), DA is discriminative since the sign denotes a clear direction. For targets that are non-negatively constrained or more level-like, especially next_volatility defined as rolling standard

deviation, DA is non-discriminative and can saturate quickly. Thus, for next_volatility, model rankings should focus on MSE/MAE, treating DA as a weak discriminative metric.

Chapter 6. Experiment Design and Hyperparameter Adjustment

6.1 Why Hyperparameter Tuning is Necessary

Adjusting hyperparameters is important because it aligns model comparisons and default settings. In this project, we take validation MSE into consideration to develop a staged tuning workflow to improve the existing working configurations.

6.2 Hyperparameter Tuning Policies

Hyperparameter setting and tuning is a multi-stage approach. Starting with the Recurrent Models, the hyperparameter settings are:
learning rate,
hidden size,
number of hidden layers,
and batch size.

For the Transformer, the order is as follows:
learning rate,
model dimension,
number of encoder layers,
number of attention heads,
and batch size.

For each stage, the best performing architecture configuration is preserved for the setting of the next hyperparameter group.

6.3 Scope of Models and Hyperparameters to be Tuned

Due to the architectural differences, recurrent models and the Transformer have distinct hyperparameter configuration space. This is done although to ensure we uniformly apply the same winner selection based on validation to all the model families.

6.4 Configurations that Yielded the Best Results

The best archive of winners pertains to task-specific tuning rather than a general global best configuration. Based on test MSE, cross-task winners are given below:

task_id	Winner	Best test MSE
sine_next_day	Baseline-LR	4.807035774110167e-08
next_return	LSTM	9.197905455948785e-05
next_volatility	Baseline-LR	1.7733482060336406e-05
next_mean_return	GRU	1.5519119187956676e-05

This indicates that the results from the tuning process are task specific and should be interpreted according to the respective task_id.

6.5 Validity Concerns About Hyperparameter Tuning

While the sequential approach to hyperparameter tuning is effective, it does not consider the range of hyperparameter combinations. The results from a later sweep can render a hyperparameter setting from an earlier stage suboptimal.

Chapter 7 - Results

In this section we present results by task, where each task is tagged by task_id and defined by target_mode + horizon (+ target_smooth_window when applicable).

7.1 Cross-task consolidated summary

Table 1 shows the summary of the best tuned winner for each task from overall_task_summary.md.

Task	task_id	target_mode	horizon	target_smooth_window	Best model (test MSE)	Best test MSE
Task A	sine_next_day	sine_next_day	1	1	Baseline-LR	4.807035774110167e-08
Task B	next_return	next_return	1	1	LSTM	9.197905455948785e-05
Task C	next_volatility	next_volatility	1	5	Baseline-LR	1.7733482060336406e-05
Task D	next_mean_return	next_mean_return	1	5	GRU	1.5519119187956676e-05

7.2 Per-task tuned comparison highlights

The results at the task level can be understood in a straightforward manner. For the synthetic sine_next_day task, all of the models do well because the target is smooth, and Baseline-LR has the lowest test MSE (4.8070e-08). This shows that for a near-linear, low-noise target, model complexity is not needed.

In the case of next_return, performance declines for all models because the target is noisier. In this case LSTM is best (9.1979e-05) and GRU is very close (9.2194e-05) whereas Baseline-LR is weaker (1.2176e-04). The best target is that gated recurrence helps on this target, but the margin is not large enough to say that there is a neural advantage.

In predicting next volatility, the consistently best performer is Baseline-LR, with a prediction of 1.7733e-05, closely followed by GRU with a prediction of 1.9411e-05. However, when predicting next mean return, the best performer is GRU, with a prediction of 1.5519e-05, followed closely by LSTM and RNN. Combined, these two tasks indicate that the ranking of models is determined by the definition of the target (raw return, smoothed return, or volatility) rather than the model class used.

7.3 Updated report artifact map

Task	task_id	Comparison report	Figures directory
Task A	sine_next_day	reports/final_report_tasks/20260331T192814Z/sine_next_day/best_tuned_comparison_sine_next_day.csv	reports/final_report_tasks/20260331T192814Z/sine_next_day/figures/
Task B	next_return	reports/final_report_tasks/20260331T192814Z/next_return/best_tuned_comparison_next_return.csv	reports/final_report_tasks/20260331T192814Z/next_return/figures/
Task C	next_volatility	reports/final_report_tasks/20260331T192814Z/next_volatility/best_tuned_comparison_next_volatility.csv	reports/final_report_tasks/20260331T192814Z/next_volatility/figures/
Task D	next_mean_return	reports/final_report_tasks/20260331T192814Z/next_mean_return/best_tuned_comparison_next_mean_return.csv	reports/final_report_tasks/20260331T192814Z/next_mean_return/figures/

7.4 Prediction pattern visualization (updated figures)\

In this section, each figure is placed right after the corresponding paragraph, with explanations included.

Task A — sine_next_day

Figure 4A-1 is included to illustrate optimization behavior and interpret model tuning quality across the simplest synthetic-style target.

Figure 4A-1. Best-tuned training-loss comparisons for sine_next_day.

- To assess if training dynamics across models are stable and consistently low.

- All models have loss trajectories that converge quickly, with minimal separation after the early epochs.
- A standalone training behavior is insufficient to identify a candidate model; validation/testing data are also required.

Figure 4A-2 is included to show which tuned model generalizes the best on the validation data for sine_next_day.

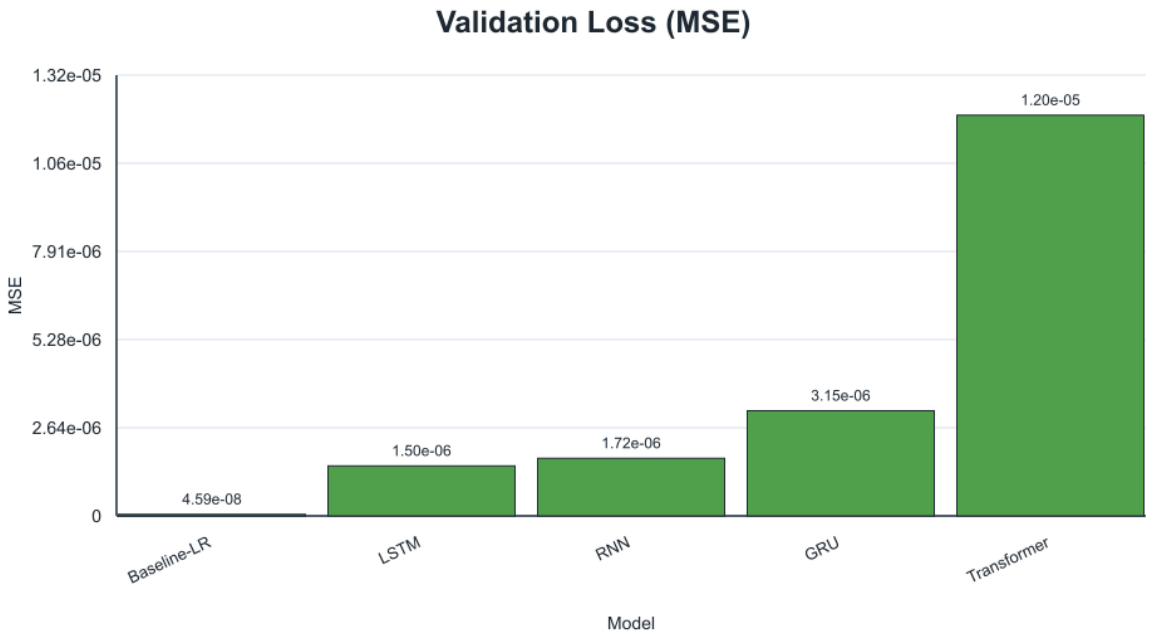

Figure 4A-2. Best tuned validation-loss results for sine_next_day.

- This is the signal that is evaluated for the model selection process while tuning.
- The validation losses are tightly clustered with the baseline and best recurrent models at the bottom.
- This validates that multiple models can generalize for this specific task which accounts for the close rankings.

Figure 4A-3 is included to assess the ranking of out-of-sample error for the held-out test split for Task A.

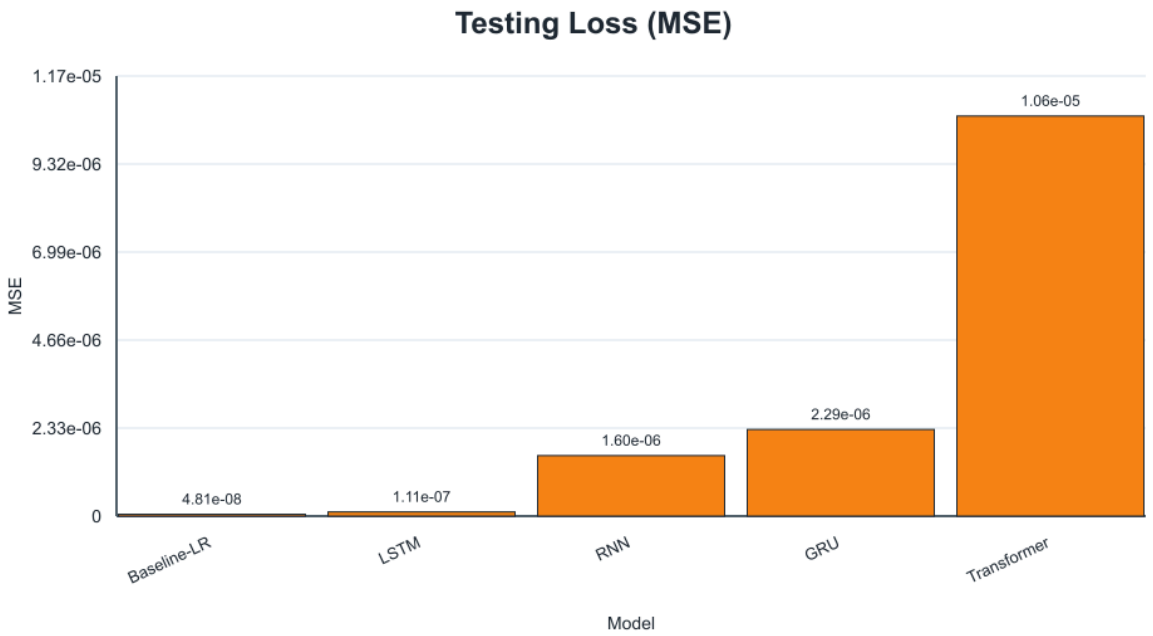

Figure 4A-3. Best tuned testing-loss results for sine_next_day.

- This evaluates whether the winners from the validation stage remain robust with completely new data.
- The baseline continues to win and is the best in the test loss category.
- The claim is supported that Baseline-LR wins Task A, even with formidable neural competitors.

Figure 4A-4 is included to determine whether stage-wise tuning had a material impact on the model performance in sine_next_day.

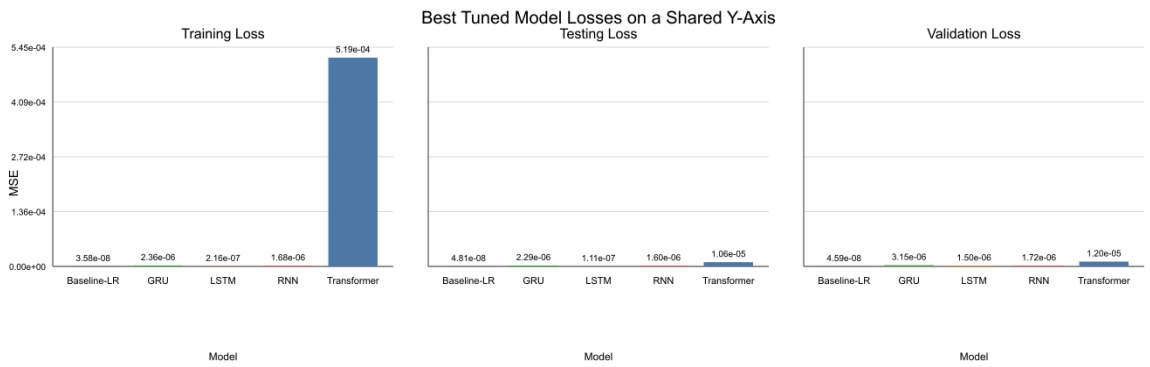

Figure 4A-4. Summary of hyperparameter impact on model losses for sine_next_day.

- This is to assess whether the tuning stages provided any 'real' benefits over the default gaps.
- There are some performance improvements, but they are minor, and after the early stages the returns fall off.
- This validates the use of constrained stage-wise tuning as adequate for this specific task.

Figure 4A-5 is included to analyze the best neural model's predictions to check for any calibration and bias.

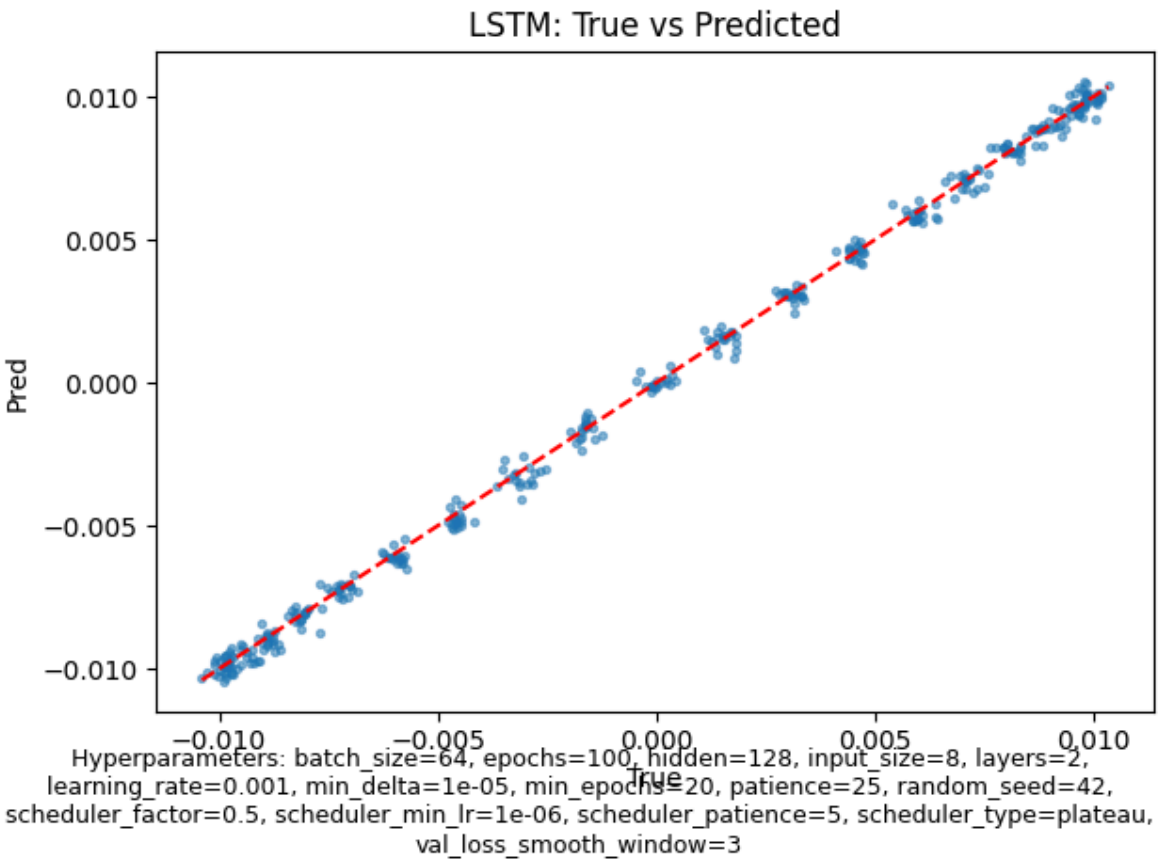

Figure 4A-5. LSTM predicted vs actual scatter for comparison for tuned sine_next_day

- Tests for linear relationship and spread in the actual vs predicted outcomes.
- Points remain clustered near the line of identity trend with minimal spread at this scale.
- Suggests the tuned LSTM is well calibrated and the only issue is the baseline error being a tad better.

Figure 4A-6 is included to evaluate the quality of temporal fit in a localized region and investigate any lag/ overshoot that occurs.

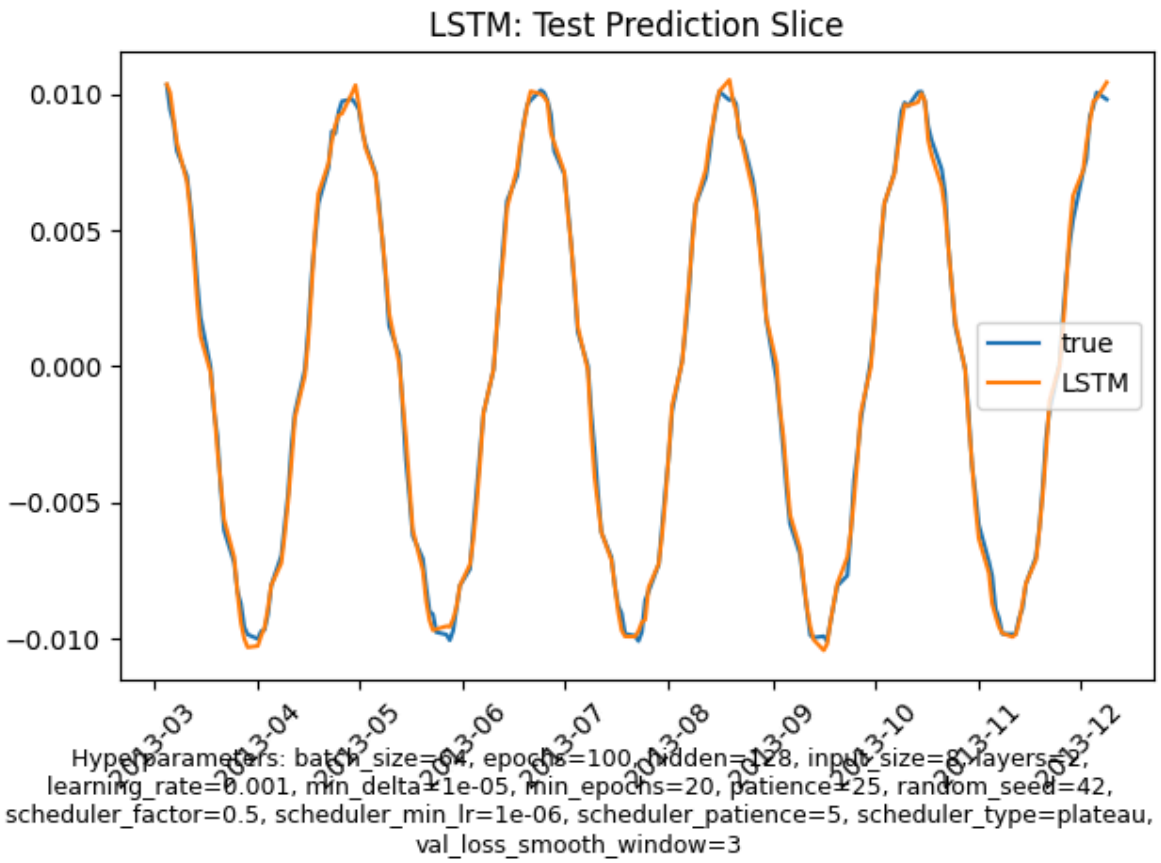

Figure 4A-6. LSTM Prediction Slice for comparison for tuned vs untuned sine_next_day.

- Confirms pointwise fit in longitudinal sense.
- Predicted and actual series are in close proximity to one another with little deviation.
- Supports the assumption that the neural network models fit well to Task A, even if other models are better in terms of MSE.

Task B — next_return

Figure 4B-1 is included for comparing training convergence behavior for the more difficult real market next_return target.

Figure 4B-1. Best Tuned Training Loss Comparison for Next Return.

- Evaluates whether all the models are optimizing the return in a stable way under a noisy return.
- All models converge, but training losses remain distinct in the end.
- Indicates that the difference in capacity/inductive-bias is more significant in this case than it was in Task A.

Figure 4B-2 is included to evaluate the generalization of validation before the final claims for Task B are tested.

Figure 4B-2. Best validated tuned validation-loss values pertaining to next_return.

- Offers the most direct influence for choosing tuned winners.
- LSTM and GRU are still just about in the best validation-loss zone, compared to the rest of the contenders.

- Validates the selection of recurrent winners in the short-list for the final out-of-sample evaluation.

Figure 4B-3 is added to support the final model ranking for next_return on the held-out test data.

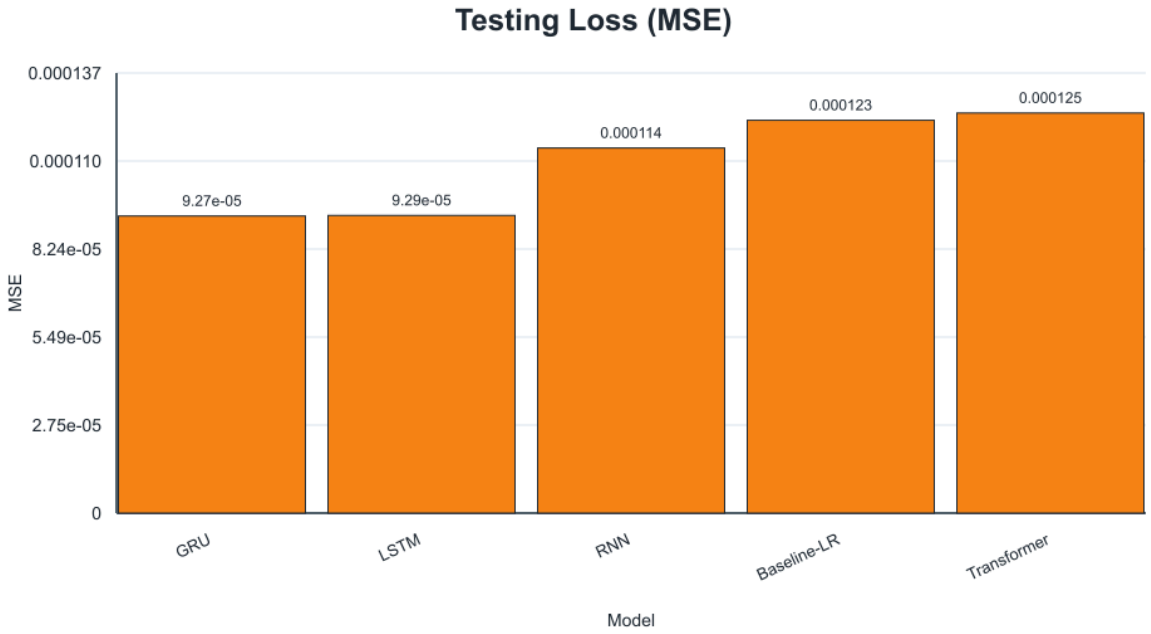

Figure 4B-3. Best tuned testing-loss values for next_return.

- Concerns if the validation ranking translates to testing.
- LSTM is the best, followed by GRU in close contention, and the baseline, though less strong, is still in the race.
- Confers the most to the claim that LSTM is the Task B winner.

Figure 4B-4 is included to analyze if the hyperparameter stages introduced meaningful improvements for each architecture.

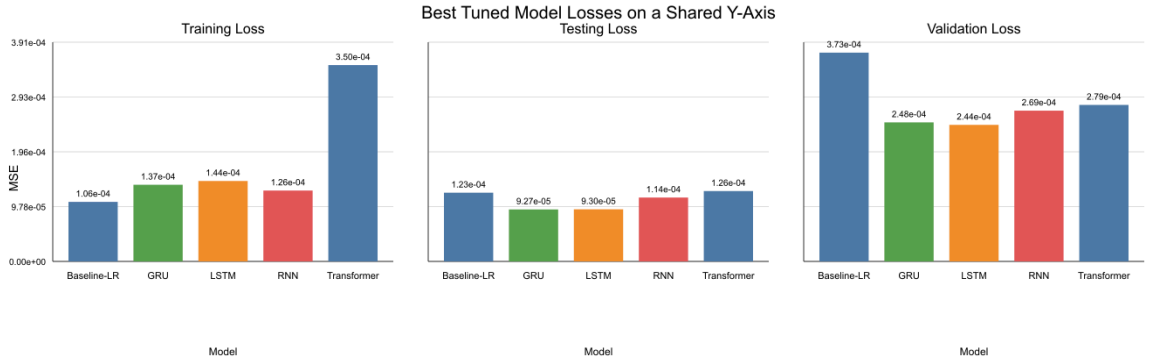

Figure 4B-4. Hyperparameter progression model-loss summary for next_return.

- Analyzes the impact of model quality in relation to the staged tuning decisions.
- Some stage decisions lead to shrinkage of loss for certain models.
- Validates rationale for emphasizing tuned results over the default comparisons.

Figure 4B-5 is added to analyze the prediction calibration of the winning LSTM on the noisy signed returns.

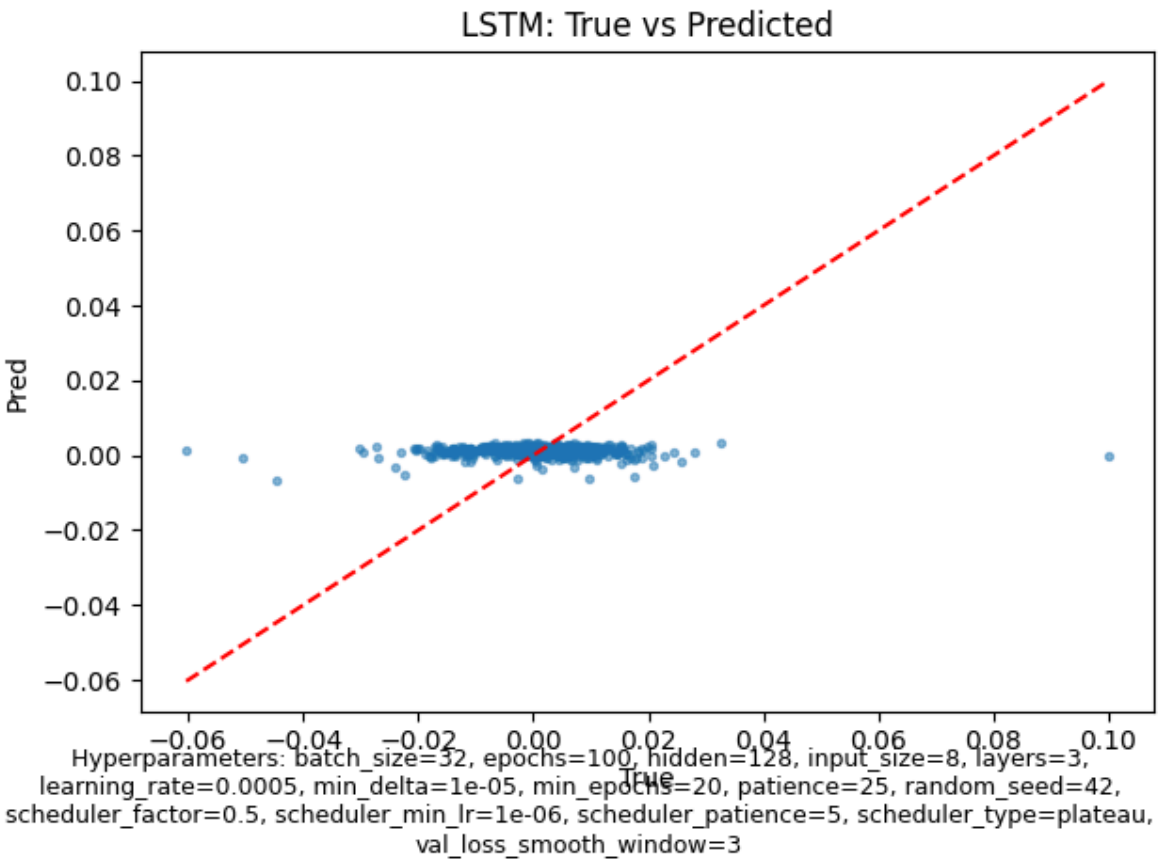

Figure 4B-5. LSTM predicted vs. actual scatter for next_return tuned comparison.

- Evaluates how winner selected is consistent and where variables are misaligned.
- Unlike Task A, this is evenly scattered with a slight trend justification.
- There is a valid predictive, yet rough and unreliable, expectation for daily returns.

Figure 4B-6 is included to evaluate point prediction and sequence level fit for selected winner.

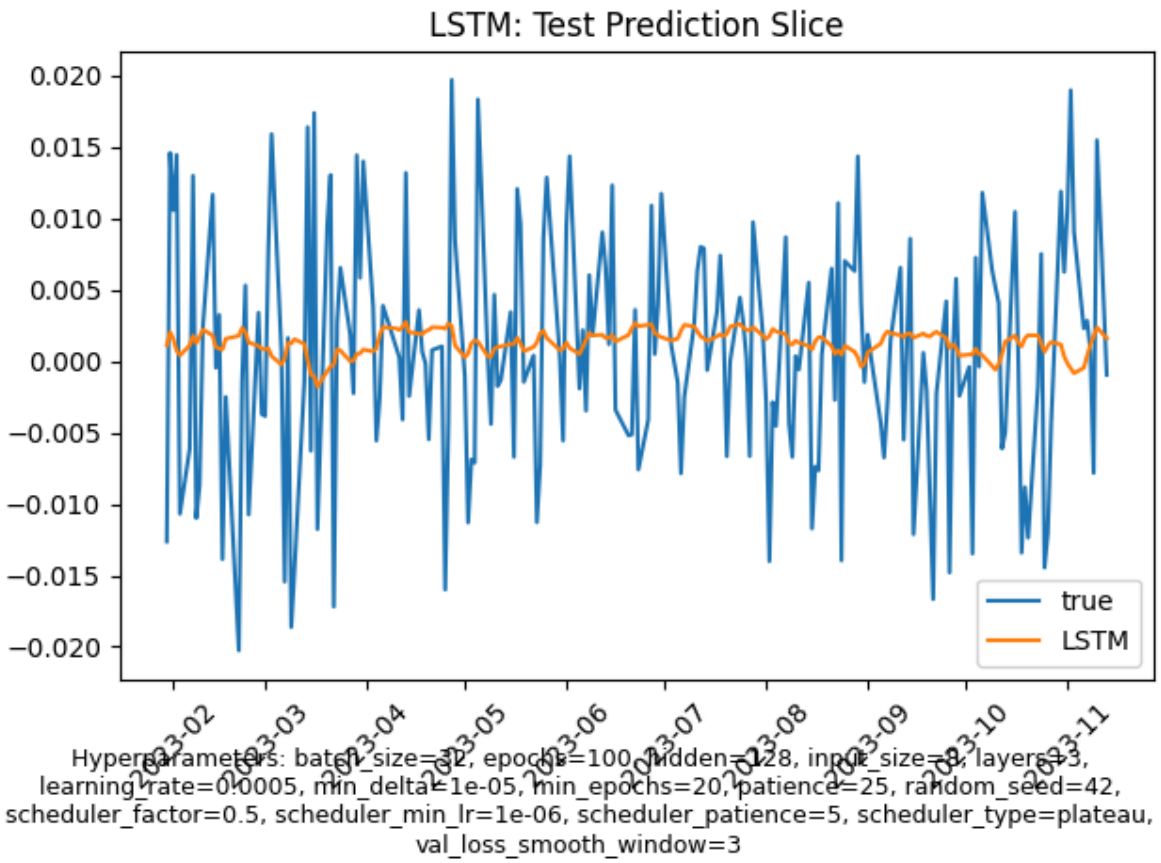

Figure 4B-6. Return prediction slice for next_return tuned comparison.

- Measures analytical predictive capability beyond aggregate metrics.
- The model does capture general movements, with a slight level of smoothing of a few sharp local movements.
- Finely granular return spikes are difficult to model, confirming the limitation.

Task C — next_volatility

Figure 4C-1 is included to assess the optimization behavior of the candidates for forecasting volatility.

Figure 4C-1. Best-tuned training loss comparison for next_volatility.

- Evaluates to what extent the models can capture the smoother, non-negative volatility target.
- The training losses are low and relatively consistent across different models.
- Suggests that this task can be effectively learned by both linear and neural models when appropriately tuned.

Figure 4C-2 is included to assess generalization quality for volatility predictions and to evaluate ranking validation.

Figure 4C-2. Best-tuned validation loss comparison for next_volatility.

- Provides the selection signal for tuned model comparison.
- runs shows bands of neural and baseline candidates at a minimum distance of loss at baseline and top candidates.

- Simpler models perform equally as well as more complex models at optimizing loss in smooth targets.

Figure 4C-3 is for final held-out ranking submissions for next_volatility

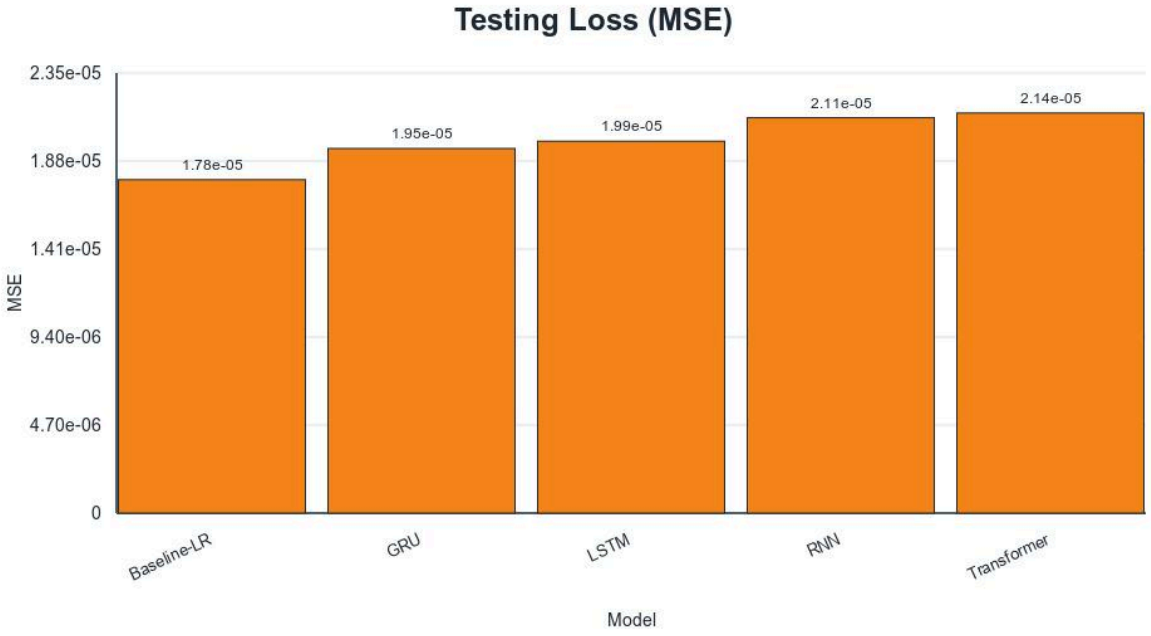

Figure 4C-3. Best-tuned testing-loss comparison for next_volatility.

- The purpose is to test if the validation results remain consistent for the next target.
- Baseline-LR attains the best test loss, while GRU is the strongest neural model.
- Baseline-LR is the best model in Task C.

Figure 4C-4 is included to assess the impact of staged hyperparameter tuning on the outcomes of the volatility modeling.

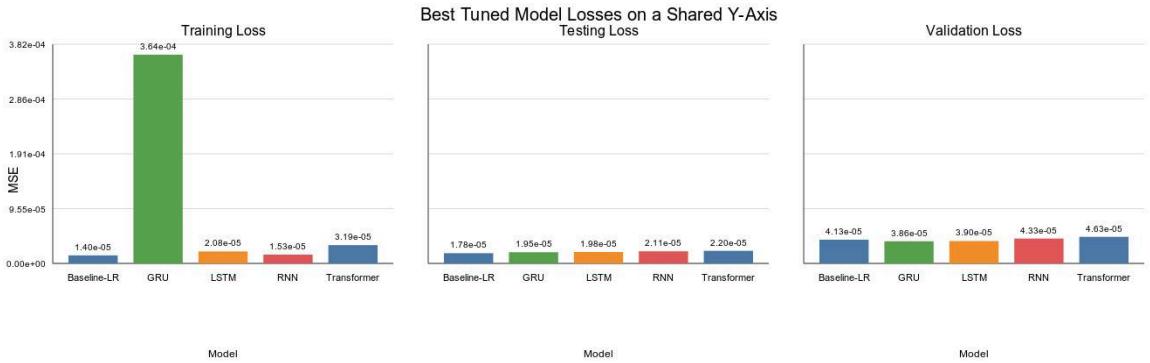

Figure 4C-4. Hyperparameter-impact model-loss summary for next_volatility.

- The purpose is to assess the anticipated benefit of each tuning stage.
- There is indeed improvement, although modest.
- Practicality of constrained tuning is supported along with the strong baseline interpretation.

Figure 4C-5 is included to assess the calibration of the best neural (GRU) predictions to the realised level of volatility for the time horizon.

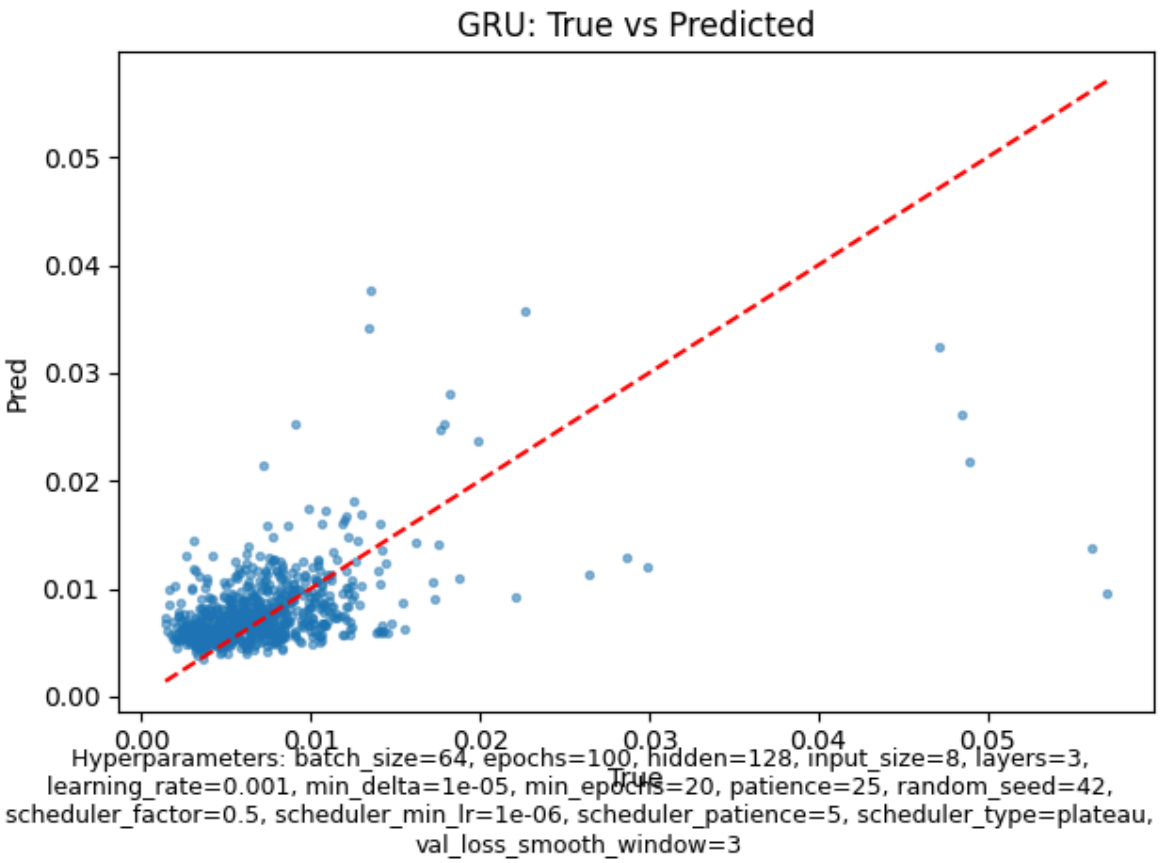

Figure 4C-5. GRU predicted-vs-actual scatter for next_volatility tuned comparison.

- focus on the central tendency and assess the level of concentration and bias surrounding the targets which are non-negative and realised.
- The predicted values are apparent, and the majority of them are along the central trend, but in the outer ranges, there is less prediction.
- There is a less extreme volatility in the outer ranges, which is a limitation.

Figure 4C-6 is included to evaluate the performance of the best neural model in tracking volatility over time.

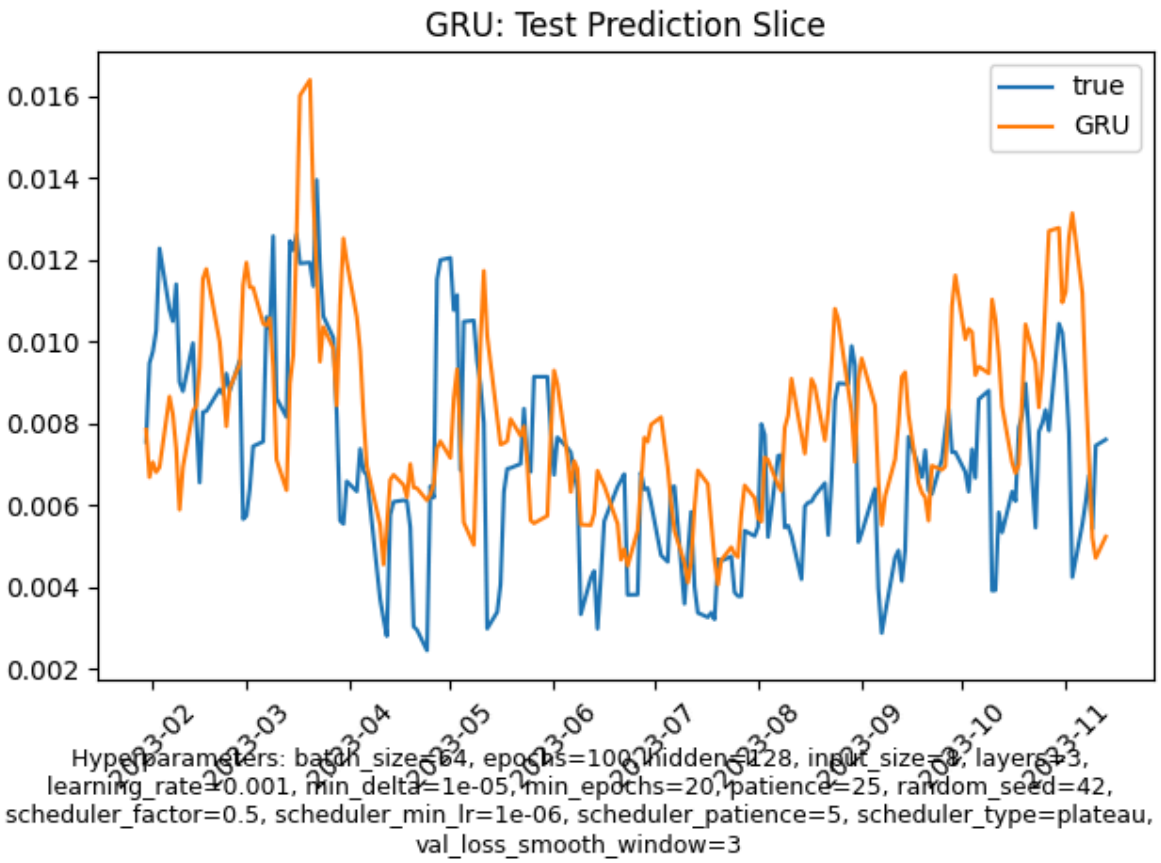

Figure 4C-6. GRU prediction slice for next_volatility tuned comparison.

- Checks whether temporal smoothing still follows major regime movements.
- The prediction curve tracks broad volatility shifts with smoother peaks.
- Supports the claim that MSE/MAE are more informative than DA for this target.

Task D — next_mean_return

Figure 4D-1 is included to compare training dynamics for moving-average return forecasting.

Figure 4D-1. Best-tuned training-loss comparison for next_mean_return.

- Tests convergence and optimisation stability across tuned models.
- Recurrent candidates reach low training losses with visible but modest separation.
- Indicates recurrent architectures have favourable fit dynamics on this smoothed target.

Figure 4D-2 is included to compare validation generalisation before reporting final test rankings.

Figure 4D-2. Best-tuned validation-loss comparison for next_mean_return.

- Provides the tuned model-selection evidence.
- GRU and nearby recurrent models occupy the lowest validation-loss range.
- Supports choosing GRU-family recurrent settings for final evaluation focus.

Figure 4D-3 is included to confirm held-out ranking for the smoothed return task.

Figure 4D-3. Best-tuned testing-loss comparison for next_mean_return.

- Tests whether selected winners maintain advantage on unseen data.
- GRU has the lowest test loss similar to LSTM and RNN.
- Supports the claim that GRU is the Task D winner.

Figure 4D-4 is included to quantify hyperparameter-stage impact on final loss outcomes.

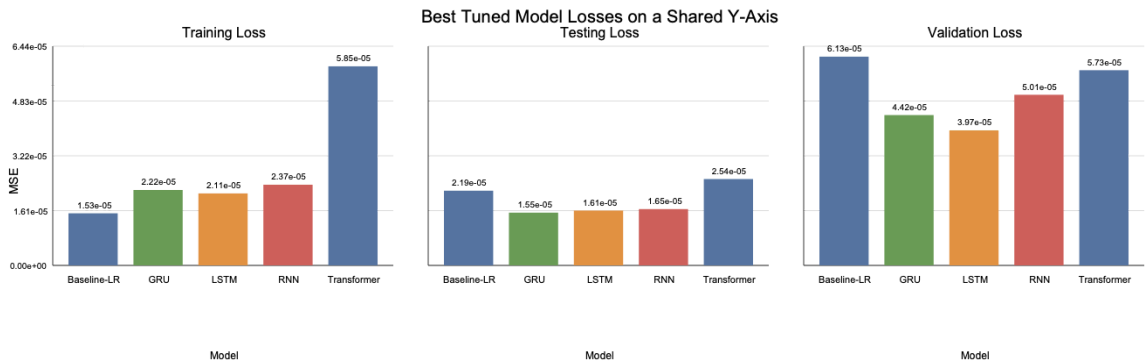

Figure 4D-4. Hyperparameter-impact model-loss summary for next_mean_return.

- Tests whether staged tuning materially improved each model family.
- Tuning improves results, with gains concentrated at specific stage transitions.
- Supports the methodological claim that stage-wise tuning was consequential.

Figure 4D-5 is included to inspect calibration quality for the GRU winner on averaged signed returns.

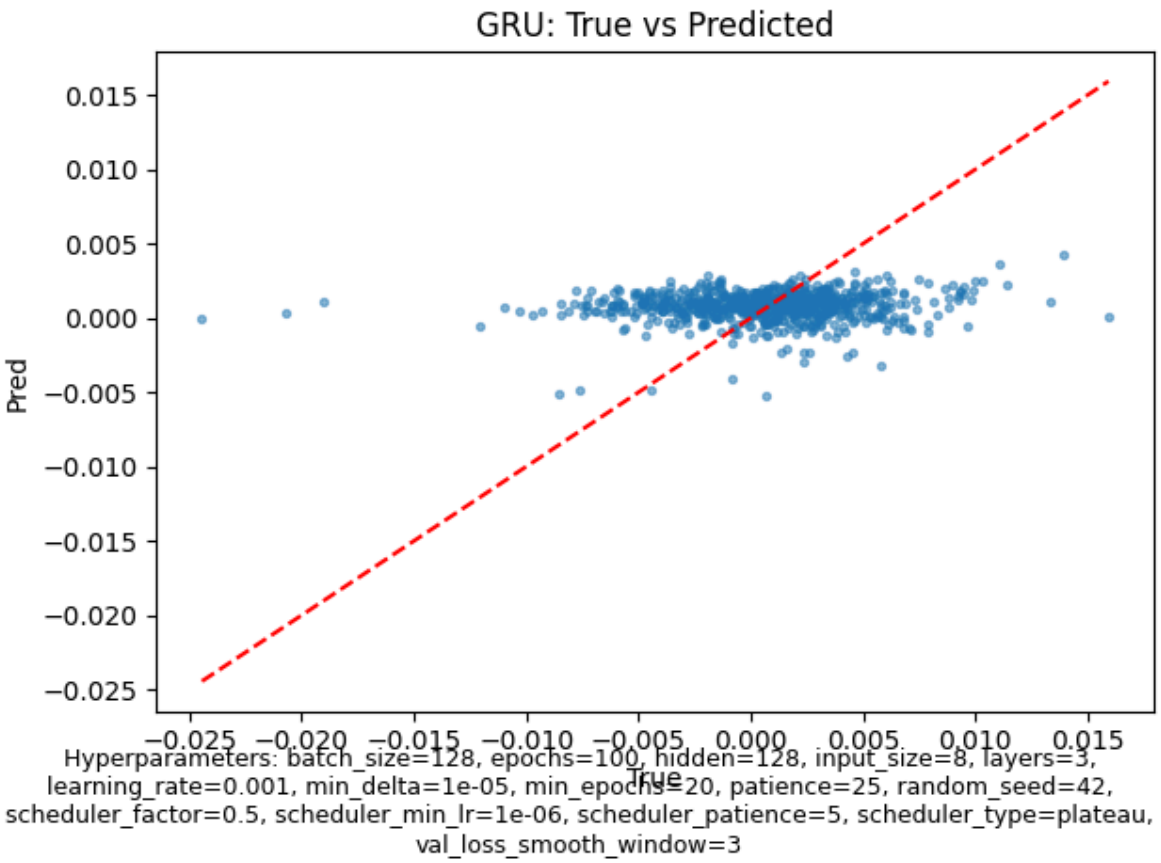

Figure 4D-5. GRU predicted-vs-actual scatter for next_mean_return tuned comparison.

- Evaluates correlation structure and spread for winner predictions.
- Points follow the central trend with moderate residual dispersion.
- Supports the claim that GRU provides the strongest overall error balance on Task D.

Figure 4D-6 is included to check time-local alignment and lag behaviour for GRU predictions.

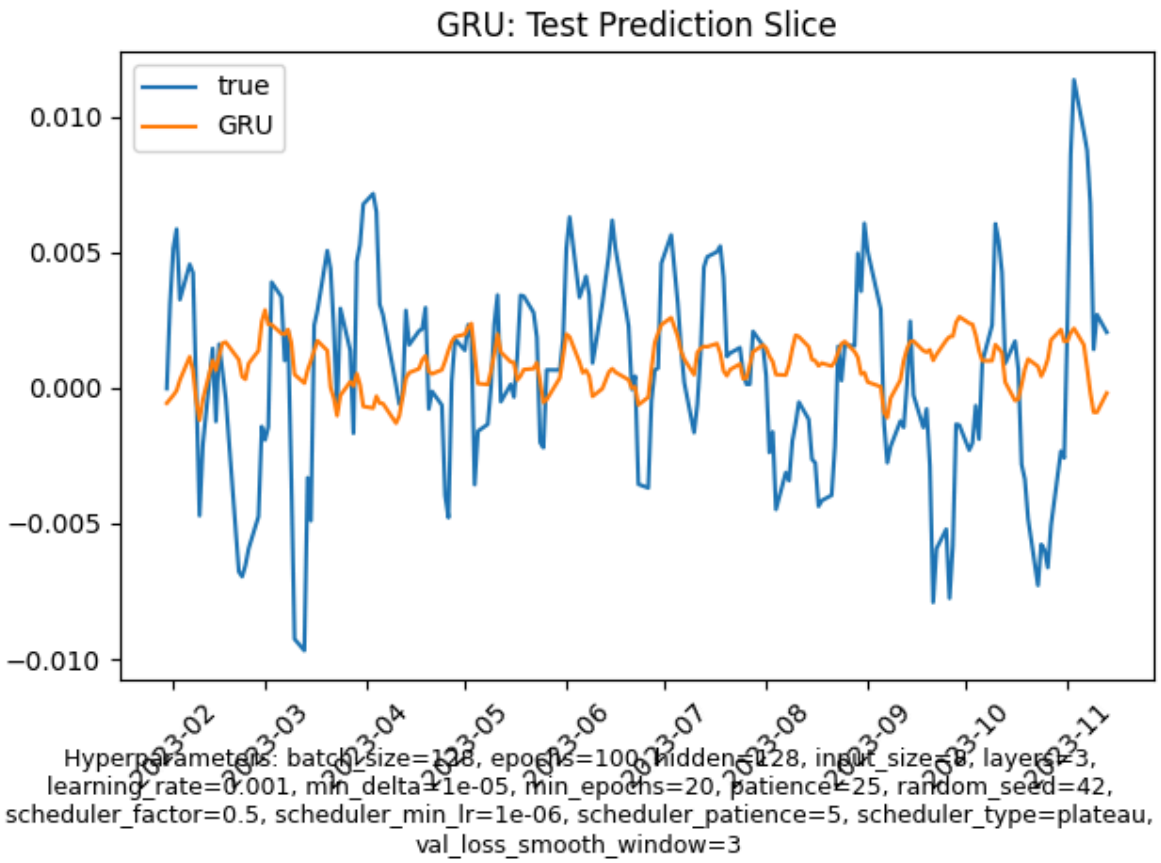

Figure 4D-6. GRU prediction slice for next_mean_return tuned comparison.

- Tests whether temporal behaviour supports quantitative winner claims.
- Predicted movement follows the direction changes with mild smoothing around peaks/troughs.
- Supports using GRU as the preferred model for next_mean_return, with expected smoothing limitations.

7.5 Summary of key findings

Key conclusions are as follows:

- No one architecture wins all tasks. Winners vary across sine_next_day, next_return, next_volatility, and next_mean_return.
- Baseline-LR is a strong benchmark, and not a formality. It is the best out of two of the four tasks and is competitive everywhere else.
- LSTM/GRU are the most competitive neural options in this analysis, but only conditionally. Their advantage seems to come when the target has enough nonlinear sequential structure to justify the increase in complexity.

Chapter 8. Discussion

The most important point of discussion is that “best model” is specific to the different tasks. We see a change in model winners with different target definitions. When the target is smoother or more or less linear in the engineered features, Baseline-LR is a tough model to beat. When the target is noisier, gated recurrent models, in particular LSTM and GRU, become better candidates, although the presence of short-memory non-linear structures still holds.

The model-specific properties described facilitate understanding of this behavior. For a reference point, Vanilla RNN is useful but it is less robust on noisy sequences. LSTM and GRU models help gated control which is likely why they perform better on next_return and next_mean_return. Given the limited scale of data and lower data dimensionality, it is reasonable that Transformer does not dominate in this case. Due to the same split strategy, scaling protocol, and tuning for all models, observed differences are more likely a function of model-task alignment.

The definition of a target must be met by metric interpretation. MSE/MAE are the main ranking metrics in the majority of cases. DA is somewhat useful for signed targets like next_return and next_mean_return, but for non-negative targets like next_volatility, it is not very useful. Starting with Baseline-LR and then switching to LSTM/GRU for a given task is a reasonable approach only if the validation and test errors turn out to be better for that task.

Chapter 9. Limitations

There are a number of limitations in this study, and they must be considered when drawing any conclusions. The real market analysis relies on a single asset proxy, SPY, so any cross-asset generalizations are unvalidated. The results come from a limited number of archived runs and not from a larger number of runs with the same seed, which means that the model gap may not be statistically significant. The benchmark is also restricted to a single-step style short-horizon formulations and a particular set of target constructions, so a different winner ordering may come from changing the horizon, the set of features, or the market regime.There are also practical reproducibility constraints. While the pipeline strives for reproducibility, the market data is sourced externally, and with time, the available historical data lengthens. This means that exact row counts and metrics will vary unless a raw snapshot is frozen. Lastly, predictive accuracy is not equal to trading accuracy. The report is devoid of the execution assumptions, transaction costs, slippage, risk limits, portfolio construction, and therefore, the forecast accuracy metrics should not be the basis for a direct profitability claim.

Chapter 10. Conclusion and Future Work

The objective of this project was to assess whether advanced neural sequence models could consistently outperform a strong linear baseline with a single controlled forecasting pipeline. The final evidence provided a more complicated answer: performance is predominantly dependent on the task. Baseline-LR was most successful on sine_next_day and next_volatility, while LSTM prevailed on next_return, and GRU excelled at next_mean_return. Ultimately, the main contribution is not a universal champion model, but rather, a strong indication that target definition (target_mode, horizon, and smoothing) significantly influences model suitability.

The work also makes a contribution for an applied training-and-reporting workflow for aligned preprocessing, leakage-safe scaling, chronological splits, staged validation driven tuning, and per task tracking artifacts. This helps to provide a clear foundation for future comparison studies and ease the audit for how each result was generated.

The work going forward needs to focus now on external validity and decision relevance through, for example, repeated-seed evaluation with uncertainty intervals, walk-forward/rolling validation, broader coverage of the assets, richer exogenous variables,

dedicated trading simulation with realistic costs and risk constraints, and realistic risk controls. Extensions with a focus on the Transformer's architecture are still valuable, but they need to be accompanied by the scale of the data and tuning budget constraints necessary for attention models to operate at the level intended.

Chapter 11. Project Contributions / What Has Been Achieved

This project was able to achieve, in the finalized consolidated report and codebase outcomes, the following:

- Creation of a coherent multi-task benchmarking workflow for sine_next_day, next_return, next_volatility, and next_mean_return.
- Model comparison standardization in the case of Baseline-LR, RNN, LSTM, GRU, and Transformer across aligned splits and metrics.
- Establishment of a reproducible final-report artifact bundle (reports/final_report_tasks/20260331T192814Z) complete with per-task diagnostics and cross-task synthesis.
- Consolidation of proposal/interim/final reporting into one self-contained final architecture with a well-defined front matter, numbered chapters, references, and a complete set of appendices.

References

[1] T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, Oct. 2018. [Online]. Available: https://doi.org/10.1016/j.ejor.2017.11.054

[2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[3] K. Cho et al., “Learning phrase representations using RNN Encoder-Decoder for statistical machine translation,” in *Proc. EMNLP*, 2014. [Online]. Available: https://arxiv.org/abs/1406.1078

[4] A. Vaswani et al., “Attention is all you need,” in *NeurIPS 2017*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/ [Online]. Available: https://github.com/robjhyndman/fpp3

[6] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: https://doi.org/10.1207/s15516709cog1402_1

[7] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning, 2021. [Online]. Available: https://www.manning.com/books/deep-learning-with-python-second-edition

[8] Z. Zhang, S. Zohren, and S. Roberts, “Deep learning for portfolio optimization,” *The Journal of Financial Data Science*, vol. 2, no. 4, pp. 8–20, 2020. [Online]. Available: https://arxiv.org/abs/2005.13665

[9] State Street Global Advisors, “SPDR S&P 500 ETF Trust (SPY),” accessed Mar. 22, 2026. [Online]. Available: https://www.ssga.com/us/en/intermediary/etfs/funds/spdr-sp-500-etf-trust-spy

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[11] R. Aroussi, “yfinance: Download market data from Yahoo! Finance’s API,” GitHub repository. [Online]. Available: https://github.com/ranaroussi/yfinance

[12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *ICLR*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

Appendices

Appendix A. Final tuned configurations (latest bundle highlights)

Task	Winner	Tuned hyperparameters	Archived run ID
sine_next_day	Baseline-LR	{"flattened_sequence": true, "model": "LinearRegression", "seq_len": 30}	best_tuned_lstm_comparison-20260331T130042Z-baseline-lr
next_return	LSTM	{"batch_size": 32, "hidden": 128, "layers": 3, "lr": 0.0005, "seq_len": 30}	best_tuned_lstm_comparison-20260331T132130Z
next_volatility	Baseline-LR	{"flattened_sequence": true, "model": "LinearRegression", "seq_len": 20}	best_tuned_lstm_comparison-20260331T134030Z-baseline-lr
next_mean_return	GRU	{"batch_size": 128, "hidden": 128, "layers": 3, "lr": 0.001, "seq_len": 30}	best_tuned_gru_comparison-20260331T135921Z

Appendix B. Additional figures

All important metrics in the archived final-report bundle are included in Section 7.4 as images in Markdown format, categorized by task (sine_next_day, next_return, next_volatility, next_mean_return) and type of figure (training/validation/testing loss, hyperparameter summary, scatter, prediction slice).

Appendix C. Reproducibility notes

The most recently archived final-report bundle used in this iteration is reports/final_report_tasks/20260331T192814Z, generated on 2026-03-31T20:31:21Z (UTC). It holds per task tuning logs, winners, tuned-comparison CSV/Markdown reports, and task-specific visuals for:

- sine_next_day
- next_return
- next_volatility
- next_mean_return

Reproducibility source-of-truth note: final reported metrics in this document emanate from runtime-effective configs (CLI overrides + staged winner promotion), as opposed to static defaults. The specific artifacts used as run truth are:

- reports/final_report_tasks/20260331T192814Z/tuning_winners.csv
- reports/final_report_tasks/20260331T192814Z/*/best_tuned_comparison_*.md
- reports/final_report_tasks/20260331T192814Z/overall_task_summary.md

The bundle root also contains overall_task_summary.csv for the same purpose of cross-task synthesis in table format.